```
        return csrfToken != null ? csrfToken.then() : Mono.empty();
      }));
      return chain.filter(exchange);
    };
  }
}
```

❶ Uses a cookie-based strategy for exchanging CSRF tokens with the Angular frontend

❷ A filter with the only purpose of subscribing to the CsrfToken reactive stream and ensuring its value is extracted correctly

Let’s verify that the logout flow works now. Ensure that the Polar UI, Redis, and Keycloak containers are still up and running. Next, build and run the application (`./gradlew bootRun`), and then go to http://localhost:9000 from an incognito browser window. Start the authentication flow by clicking the Login button from the upper-right menu. Then click the Logout button. Under the hood, Spring Security will now accept your logout request (Angular adds the CSRF token value from the cookie as an HTTP header), terminate your web session, propagate the request to Keycloak, and finally redirect you to the homepage, unauthenticated.

Thanks to this change, you can also perform any `POST`, `PUT`, and `DELETE` requests without receiving a CSRF error. Feel free to explore the Angular application. If you start up Catalog Service and Order Service, you can try adding new books to the catalog, modifying them, or placing an order.

Both Isabelle and Bjorn can perform any action at the moment, which is not what we want, since customers (like Bjorn) shouldn’t be allowed to manage the book catalog. The next chapter will cover authorization, and you’ll see how to protect each endpoint with different access policies. Before addressing authorization, though, we need to write autotests to cover the new functionality. That’s coming up in the next section.